Nominal Types / Random Access Lists

Mid-term exam, MPRI FUN

November 26, 2025 — Duration: 2h45

Answers are judged by their correctness, but also by their clarity and conciseness. An answer need not be accompanied with a proof unless a proof is explicitly requested.

Although the questions are in English, it is permitted to answer in French or English. It is recommended to answer in French if this is your mother tongue.

This assignment is divided in two independent parts. The two parts can be treated in any order. The first part is concerned with the meta-theory of a type system and involves writing (short) proofs. The second part is concerned with functional programming in OCaml and involves writing (short) programs.

1 Nominal Types

In this part, we study an extension of the simply-typed λ-calculus with *nominal types*, also known as *iso-recursive types*.

1.1 Basic Definitions

We assume that there is an infinite set of data type *names*. Concrete examples of names could be *bool*, *list*, *tree*, etc. We let n range over names.

The syntax of terms is:

$$t ::= x \mid \lambda x.t \mid t\ t \mid \mathsf{fold}_n\, t \mid \mathsf{unfold}_n\, t$$

The syntax of values and evaluation contexts is:

$$\begin{array}{rcl} v & ::= & \lambda x.t \mid \mathsf{fold}_n\, v \\ E & ::= & [\,] \mid E\ t \mid v\ E \mid \mathsf{fold}_n\, E \mid \mathsf{unfold}_n\, E \end{array}$$

The reduction rules are:

$$\begin{array}{rcll} (\lambda x.t)\ v & \longrightarrow & t[v/x] & (\beta_v) \\ \mathsf{unfold}_n\,(\mathsf{fold}_n\, v) & \longrightarrow & v & (\mathit{unfold}/\mathit{fold}) \\ E[t] & \longrightarrow & E[t'] \quad \text{if } t \longrightarrow t' & (\mathit{context}) \end{array}$$

Question 1 *Give an example of a closed stuck term, that is, a closed term that is not a value and that cannot be reduced.* Answer □

A type name n is considered as a type. Therefore, the syntax of types is:

$$T ::= X \mid n \mid T \to T$$

We allow the user to give a set $\mathcal{D}$ of *nominal type definitions* of the form $\mathsf{type}\ n \equiv T$. To indicate the existence of such a definition, we write $(\mathsf{type}\ n \equiv T) \in \mathcal{D}$.

Question 2 *In this question, we assume that the syntax of types is extended with sums and products of arity 0 and 2: that is,* $T ::= \ldots \mid 0 \mid 1 \mid T + T \mid T \times T$. *In this setting,*

1. *Propose a nominal type definition for the type "bool" of Booleans.*
2. *Propose a nominal type definition for the type "blist" of lists of Booleans.*

Answer □

The syntax of type environments is $\Gamma ::= \emptyset \mid \Gamma; x : T$.
We write $\Gamma; \Gamma'$ for the concatenation of the type environments Γ and Γ'.
The typing judgement is inductively defined by the following rules:

$$\textsc{Var}\quad \Gamma \vdash x : \Gamma(x) \qquad \textsc{Abs}\quad \frac{\Gamma; x : T_1 \vdash t : T_2}{\Gamma \vdash \lambda x.t : T_1 \to T_2} \qquad \textsc{App}\quad \frac{\Gamma \vdash t_1 : T_1 \to T_2 \qquad \Gamma \vdash t_2 : T_1}{\Gamma \vdash t_1\ t_2 : T_2}$$

$$\textsc{Fold}\quad \frac{\Gamma \vdash t : T \qquad (\mathsf{type}\ n \equiv T) \in \mathcal{D}}{\Gamma \vdash \mathsf{fold}_n\ t : n} \qquad \textsc{Unfold}\quad \frac{\Gamma \vdash t : n \qquad (\mathsf{type}\ n \equiv T) \in \mathcal{D}}{\Gamma \vdash \mathsf{unfold}_n\ t : T}$$

Question 3 *In this question, we again assume that the syntax of types is extended with sums and products: that is to say,* $T ::= \ldots \mid 0 \mid 1 \mid T + T \mid T \times T$. *We also assume that the syntax of values is extended with the unit value, injections, and pairs: that is,* $v ::= \ldots \mid () \mid \mathit{inj}_i\ v \mid (v, v)$, *where* $i \in \{1, 2\}$. *We assume that the type system is extended with the usual typing rules for unit, sums, and products. In this setting,*

1. *Give two closed values that inhabit the type bool that you proposed in Question 2.*
2. *Give two closed values that inhabit the type blist that you proposed in Question 2.*

Answer □

1.2 Type Soundness

In this section, we come back to the pure λ-calculus as defined in the text above, outside of Questions 2 and 3. That is, we consider a λ-calculus *without* sums and products.

In the course, this Value Substitution lemma was mentioned and used:

Lemma 1 $x : T_1; \Gamma \vdash t : T_2$ *and* $x \notin dom(\Gamma)$ *and* $\emptyset \vdash v : T_1$ *imply* $\Gamma \vdash t[v/x] : T_2$.

Question 4 *Does this lemma still hold in the presence of nominal types? What is the general structure of its proof? How is this proof affected by the introduction of nominal types? A brief answer is expected; no proof is requested.* Answer □

In the course, this Subject Reduction lemma was proved:

Lemma 2 $\emptyset \vdash t : T$ *and* $t \longrightarrow t'$ *imply* $\emptyset \vdash t' : T$.

We wish to prove that this lemma still holds in the presence of nominal types. The proof is by structural induction on the second hypothesis, that is, $t \longrightarrow t'$. Thus, there is one case for each reduction rule.

Question 5 *Give the proof of the first case of Subject Reduction, namely the case* (β_v). *Emphasize the places where the proof changes (due to the introduction of nominal types) compared to the proof that was studied in the course; or, if the proof does not change, emphasize the reasons why it does not change.* Answer □

Question 6 *Give the proof of the second case of Subject Reduction, namely (unfold/fold).* Answer □

In the course, the following Classification lemma was given:

Lemma 3 *Assume $\emptyset \vdash v : T$. Then,*

- *if T is an arrow type, then v is of the form $\lambda x.t$;*
- *...*

Question 7 *In the presence of nominal types, write down the full statement of this lemma, and provide a proof of this lemma.* Answer □

In the course, the following Progress lemma was proved:

Lemma 4 *If $\emptyset \vdash t : T$ then either $\exists t'.\ t \longrightarrow t'$ or t is a value.*

We wish to prove that this lemma still holds in the presence of nominal types. The proof is by structural induction on the hypothesis $\emptyset \vdash t : T$. Thus, there is one case for each typing rule. We shall admit that the cases that have been studied in the course (VAR, ABS, APP) are unchanged.

Question 8 *Give the two new cases of the proof of Progress, namely FOLD and UNFOLD.* Answer □

Question 9 *Is it dangerous to allow the user to give two definitions* $\mathsf{type}\ n \equiv T_1$ *and* $\mathsf{type}\ n \equiv T_2$ *regarding the same name n? If so, where is this danger evident in the proof of the Subject Reduction or Progress lemmas? If not, why is it not dangerous?* Answer □

Question 10 *Is it dangerous to allow the user to give a circular definition* $\mathsf{type}\ n \equiv n$*? If so, where is this danger evident in the proof of the Subject Reduction or Progress lemmas? If not, why is it not dangerous?* Answer □

1.3 Erasure

This section contains only one question, namely Question 11. The question is not difficult, we believe, but a complete answer can be long. You may prefer to move on to section 2.

We have introduced two new constructs in the syntax of terms, namely $\mathsf{fold}_n\, t$ and $\mathsf{unfold}_n\, t$. We would now like to prove that these constructs do not influence the behavior of well-typed programs.

We wish to define the *erasure* of a term t to be the same term, deprived of any fold or unfold annotations. We write $\lceil t \rceil$ for the erasure of the term t.

Question 11 *Give a formal definition of erasure, a function of terms to terms. Then, prove that:*

- *If t reduces (in zero, one, or many steps) to a value then the same is true of $\lceil t \rceil$.*
- *If t diverges then the same is true of $\lceil t \rceil$.*

This proof may require auxiliary definitions or auxiliary lemmas. Please provide the necessary definitions, lemmas, and proofs, while remaining concise. If a fact is "obvious" then it is acceptable to state this fact, to briefly say how one would prove it, and to omit its proof. Answer □

2 Binary Random Access Lists

In this part, we study an immutable data structure, *binary random access lists*. A binary random access list represents a sequence of elements. Binary random access lists offer efficient insertion and extraction at the front of the sequence (`cons`, `uncons`) as well as efficient random access, that is, efficient access to the i-th element of the sequence. In the following, we call them just *sequences*, for short.

Sequences are defined in OCaml as follows:

```
type 'a seq =
  | NIL
  | ZERO of      ('a * 'a) seq
  | ONE  of 'a * ('a * 'a) seq
```

There is a strong analogy between the structure of a sequence and the structure of a natural number in binary notation.

By convention, when writing down a natural number n in binary notation, let us write the least significant digits first (at the left end) and the most significant digits last (at the right end). So, for example, the number "two" is written 01, the number "three" is written 11, and the number "six" is written 011. The number "zero" is written as the empty sequence of digits, ϵ.

Then, in a sequence of n elements, the data constructors `ZERO` and `ONE` appear exactly in the same order as the digits 0 and 1 in the binary representation of the number n. In other words, the data constructors `NIL`, `ZERO` and `ONE` respectively correspond to the empty sequence ϵ, the digit 0, and the digit 1. By convention, `ZERO NIL` is forbidden: that is, the argument of the data constructor `ZERO` must never be `NIL`.

In the following, please write down the type and the code of each function as you would in OCaml, in a form that is as concise and elegant as possible. Regarding syntax, perfection is not required: we will judge the spirit, not the letter. The essential point is that the code should be well-typed and correct. We recall that in OCaml, when a function is polymorphic recursive, its polymorphic type must be explicitly given.

Question 12 *Write down the OCaml values of type* `int seq` *that represent the following sequences of integers:*

- *1, 2;*
- *1, 2, 3;*
- *1, 2, 3, 4, 5, 6.*

Answer □

Question 13 *Write a function* `cons` *which inserts an element in front of a sequence. In other words,* `cons x xs` *should be a sequence whose first element is* `x` *and whose remaining elements are the elements of* `xs`*. Justify why the code never builds a sequence of the form* `ZERO NIL`*.* Answer □

Question 14 *Write a function* `uncons` *which extracts an element at the front of a non-empty sequence. In other words,* `uncons xs` *should be a pair of the first element of the sequence* `xs` *and a sequence of the remaining elements of* `xs`*. Justify why the code never builds a sequence of the form* `ZERO NIL`*. Hint: there is a certain symmetry between* `cons` *and* `uncons`*.* Answer □

Question 15 *What is the worst-case time complexity of the functions* `cons` *and* `uncons` *that you have proposed in Question 13 and Question 14? A brief answer and justification is expected.* Answer □

Question 16 *Write a function* `get` *which reads an element at an arbitrary index in a sequence. In other words, provided* `i` *is comprised between 0 (included) and the length of the sequence* `xs` *(excluded),* `get i xs` *should return the* `i`*-th element of the sequence* `xs`*. We assume that* `i` *is a machine integer: its type in OCaml is* `int`*. You may use OCaml's built-in operations on integers, including addition, subtraction, multiplication, division, modulo (*`x mod y`*), and so on.* Answer □

Question 17 *Write a function* `set` *which replaces an element at an arbitrary index in a sequence with a new element. In other words, provided* `i` *is comprised between 0 (included) and the length of the sequence* `xs` *(excluded),* `set i e xs` *should return a sequence whose length is the length of* `xs`*, whose* `i`*-th element is* `e`*, and whose* `j`*-th element is the* `j`*-th element of* `xs`*, when* `i` *and* `j` *differ. Hint: you may call* `cons`*,* `uncons`*, or* `get`*. Justify why the code never builds a sequence of the form* `ZERO NIL`*.* Answer □

Question 18 *What is the worst-case time complexity of the functions* `get` *and* `set` *that you have proposed in Question 16 and Question 17? A brief answer and justification is expected.* Answer □

We now define a function `transform`, which is analogous to `set`, but is slightly more flexible than `set`. Instead of replacing the `i`-th element of the sequence with a fixed value, it transforms the `i`-th element of the sequence by applying a transformation function `f` to it. This function is defined as follows:

```
let rec transform : 'a . int -> ('a -> 'a) -> 'a seq -> 'a seq =
fun i f xs ->
  match xs with
  | NIL ->
      assert false
  | ZERO ps ->
      ZERO (transform_ZERO i f ps)
  | ONE (x, ps) ->
      if i = 0 then ONE (f x, ps)
      else ONE (x, transform_ZERO (i-1) f ps)

and transform_ZERO : 'a. int -> ('a -> 'a) -> ('a * 'a) seq -> ('a * 'a) seq =
fun i f ps ->
  let f =
    if i mod 2 = 0 then
      fun (x, y) -> (f x, y)
    else
      fun (x, y) -> (x, f y)
  in
  transform (i / 2) f ps
```

Question 19 *Give a new definition of* `set` *in one line using* `transform`*.* Answer □

Question 20 *By applying a selective defunctionalization to* `transform` *and to* `set` *(as defined in Question 19), propose new versions of* `transform` *and* `set` *where* `transform`

still takes three parameters `i`*,* `f`*, and* `xs` *and where* `f` *is a data structure instead of a function.* Answer □

Question 21 *What is the worst-case time complexity of* `transform`*? By this, we mean either the version of* `transform` *that we have given, or the variant of* `transform` *that you have proposed in Question 20.* Answer □

3 Solutions

Question 1

Two kinds of examples come to mind. The first kind is a function application whose left-hand side is not a λ-abstraction: an example is $(\mathsf{fold}_n\, v_1)\; v_2$, where v_1 and v_2 are values. The second kind is a use of unfold_n with an unsuitable argument: examples are $\mathsf{unfold}_n\, (\lambda x.t)$ and $\mathsf{unfold}_{n_1}\, (\mathsf{fold}_{n_2}\, v)$ where n_1 and n_2 are distinct names.

Question 2

There should be two Boolean values, corresponding to "false" and "true". A type with two inhabitants, such as $1 + 1$, reflects this intent. Thus, the nominal type of Booleans can be defined by $\mathsf{type}\ \mathit{bool} \equiv 1 + 1$.

A list is either empty or formed of one element and another list. Thus, the nominal type of lists of Booleans can be defined by $\mathsf{type}\ \mathit{blist} \equiv 1 + (\mathit{bool} \times \mathit{blist})$.

Question 3

The two inhabitants of the type 1+1 are $\mathit{inj}_1\ ()$ and $\mathit{inj}_2\ ()$. Therefore, the two inhabitants of the type *bool* are $\mathit{false} \triangleq \mathit{fold}_{\mathit{bool}}\ (\mathit{inj}_1\ ())$ and $\mathit{true} \triangleq \mathit{fold}_{\mathit{bool}}\ (\mathit{inj}_2\ ())$.

Similarly, the empty list is the value $\mathit{nil} \triangleq \mathit{fold}_{\mathit{blist}}\ (\mathit{inj}_1\ ())$. An example of a list of one element is the value $\mathit{fold}_{\mathit{blist}}\ (\mathit{inj}_2\ (\mathit{false}, \mathit{nil}))$.

Question 4

Yes, the lemma still holds; its statement does not need to be modified. The proof is by structural induction on the first hypothesis, that is, on the typing judgement $x : T_1; \Gamma \vdash t : T_2$. There is one case per typing rule, that is, three cases in the pure λ-calculus (Var, Abs, App), and two more cases once nominal types are introduced (Fold, Unfold).

Question 5

In the case of (β_v), the first hypothesis is

$$\emptyset \vdash (\lambda x.t)\; v : T_2$$

and the goal is

$$\emptyset \vdash t[v/x] : T_2$$

Despite the introduction of nominal types, the type system is still syntax-directed: there is one typing rule for each construct in the syntax of terms. Therefore, the derivation of the first hypothesis must be of this form, for some type T_1:

$$\textsc{App}\ \frac{\textsc{Abs}\ \dfrac{x : T_1 \vdash t : T_2}{\emptyset \vdash \lambda x.t : T_1 \to T_2} \qquad \emptyset \vdash v : T_1}{\emptyset \vdash (\lambda x.t)\; v : T_2}$$

Then, the goal is a direct consequence of the Value Substitution lemma (Lemma 1). In summary, because the type system is still syntax-directed, this proof case does not change at all.

Question 6

In the case of (*unfold*/*fold*), the first hypothesis is

$$\emptyset \vdash \mathsf{unfold}_n\,(\mathit{fold}_n\ v) : T$$

and the goal is

$$\emptyset \vdash v : T$$

The derivation of the first hypothesis must be of this form:

$$\textsc{Unfold}\ \frac{\textsc{Fold}\ \dfrac{\emptyset \vdash v : T' \qquad (\mathsf{type}\ n \equiv T') \in \mathcal{D}}{\emptyset \vdash \mathit{fold}_n\ v : n} \qquad (\mathsf{type}\ n \equiv T) \in \mathcal{D}}{\emptyset \vdash \mathsf{unfold}_n\,(\mathit{fold}_n\ v) : T}$$

While writing this derivation, we have not made any assumption about the set of type definitions $\mathcal{D}$. In general, this set might contain two distinct definitions for the name n, which is why we have not assumed, a priori, that the types T and T' are equal. Let us now assume that the set $\mathcal{D}$ contains at most one definition for each name n. (This assumption is indeed necessary, as suggested by Question 9). Then T' and T must be the same type. Therefore, the first premise of Fold, namely $\emptyset \vdash v : T$, is the goal.

Question 7

The statement must be completed by writing:

- if T is a nominal type n, then v is of the form $\mathsf{fold}_n\ v'$.

A small yet important detail is that the index n in $\mathsf{fold}_n\ v$ is determined by the type T. In other words, the following alternative statement is also correct, but too weak:

- if T is a nominal type then v is of the form $\mathsf{fold}_n\ v'$.

The proof (of the stronger statement) is by cases on the hypothesis $\emptyset \vdash v : T$. (Induction is not required.) Considering that v is a value and is closed (it is well-typed under an empty type environment), the hypothesis $\emptyset \vdash v : T$ must be a consequence of either Abs or Fold. In each case, the result is immediate.

Question 8

In the case of Fold, the hypothesis is

$$\textsc{Fold}\ \frac{\emptyset \vdash t : T \qquad (\mathsf{type}\ n \equiv T) \in \mathcal{D}}{\emptyset \vdash \mathsf{fold}_n\ t : n}$$

and the goal is to show that $\mathsf{fold}_n\ t$ is either reducible or a value. By the induction hypothesis, applied to $\emptyset \vdash t : T$, the term t is either reducible or a value. If t is reducible then, because $\mathsf{fold}_n\ [\,]$ is an evaluation context, $\mathsf{fold}_n\ t$ is reducible as well. If t is a value v then $\mathsf{fold}_n\ t$ is a value $\mathsf{fold}_n\ v$.

In the case of Unfold, the hypothesis is

$$\textsc{Unfold}\ \frac{\emptyset \vdash t : n \qquad (\mathsf{type}\ n \equiv T) \in \mathcal{D}}{\emptyset \vdash \mathsf{unfold}_n\ t : T}$$

and the goal is to show that $\mathsf{unfold}_n\ t$ is either reducible or a value. By the induction hypothesis, applied to $\emptyset \vdash t : n$, the term t is either reducible or a value. If t is reducible then, because $\mathsf{unfold}_n\ [\,]$ is an evaluation context, $\mathsf{unfold}_n\ t$ is reducible as well. If t is

a value then the Classification lemma, applied to the hypothesis $\emptyset \vdash t : n$, implies that t must be a value of the form $\mathsf{fold}_n\, v$. Therefore, the term $\mathsf{unfold}_n\, t$ is $\mathsf{unfold}_n\, (\mathsf{fold}_n\, v)$, which reduces to v, therefore is reducible.

Question 9

Yes, it is dangerous. This danger is evident in the answer to Question 6, where we have had to assume that there is at most one definition of each nominal type n. If one could give two definitions of a nominal type n then one could fold at type T_1 and unfold at type T_2; therefore one would be able to convert a value of type T_1 to a value of type T_2. This would break Subject Reduction and Type Soundness.

Question 10

No, it is not dangerous. Type Soundness can be proved even in the presence of such a circular definition.

Optionally, one can make the following remark. If a type n is defined by $\mathsf{type}\ n \equiv n$ then it is effectively an empty type. An intuition is that the type n is empty because a closed value that inhabits this type would have to be the infinite value $\mathsf{fold}_n\, (\mathsf{fold}_n\, (\cdots))$. Because values are finite, this is impossible. More formally, one can prove that $\emptyset \vdash v : n$ implies a contradiction. The proof is by structural induction on the hypothesis $\emptyset \vdash v : n$. By inspection of the typing rules, the value v must be of the form $\mathsf{fold}_n\, v'$ and the derivation of the hypothesis $\emptyset \vdash v : n$ must be of this form:

$$\textsc{Fold}\ \frac{\emptyset \vdash v' : n \qquad (\mathsf{type}\ n \equiv n) \in \mathcal{D}}{\emptyset \vdash \mathsf{fold}_n\, v' : n}$$

The induction hypothesis, applied to the judgement $\emptyset \vdash v' : n$, yields a contradiction.

Question 11

Erasure is inductively defined as follows:

$$\begin{array}{rcl} \lceil x \rceil & = & x \\ \lceil \lambda x.t \rceil & = & \lambda x.\lceil t \rceil \\ \lceil t_1\ t_2 \rceil & = & \lceil t_1 \rceil\ \lceil t_2 \rceil \\ \lceil \mathsf{fold}_n\, t \rceil & = & \lceil t \rceil \\ \lceil \mathsf{unfold}_n\, t \rceil & = & \lceil t \rceil \end{array}$$

Obviously, erasure commutes with substitution: that is, $\lceil t[v/x] \rceil$ is $\lceil t \rceil [\lceil v \rceil / x]$. This can be proved by structural induction on the term t. All cases are straightforward.

Obviously, the erasure of a value is a value: that is, $\lceil v \rceil$ is a value. This can be proved by structural induction on the value v. The erasure of a λ-abstraction is a λ-abstraction, which is a value. The erasure of $\mathsf{unfold}_n\, v$ is the erasure of v, which by the induction hypothesis is a value.

It is convenient to define the erasure of an evaluation context as follows:

$$\begin{array}{rcl} \lceil [\,] \rceil & = & [\,] \\ \lceil E\ t \rceil & = & \lceil E \rceil\ \lceil t \rceil \\ \lceil v\ E \rceil & = & \lceil v \rceil\ \lceil E \rceil \\ \cdots & = & \cdots \end{array}$$

Obviously, the erasure of an evaluation context is an evaluation context, and erasure commutes with context filling: that is, $\lceil E[t] \rceil$ is $\lceil E \rceil [\lceil t \rceil]$. This can be proved by structural induction on the context E.

One defines the *size* of a term as the number of fold and unfold annotations that it contains. An inductive definition is as follows:

$$\begin{aligned} size(x) &= 0 \\ size(\lambda x.t) &= size(t) \\ size(t_1\ t_2) &= size(t_1) + size(t_2) \\ size(\mathsf{fold}_n\, t) &= size(t) + 1 \\ size(\mathsf{unfold}_n\, t) &= size(t) + 1 \end{aligned}$$

Obviously, on can define also the size of an evaluation context, $size(E)$, so that size commutes with context filling: that is, $size(E[t])$ is $size(E) + size(t)$.

One can then prove the following Forward Simulation lemma: if $t \longrightarrow t'$ then

- either $\lceil t \rceil \longrightarrow \lceil t' \rceil$, that is, $\lceil t \rceil$ reduces to $\lceil t' \rceil$ in one step;
- or $\lceil t \rceil = \lceil t' \rceil$ and $size(t) > size(t')$.

The proof is by structural induction on the hypothesis $t \longrightarrow t'$. The three cases are as follows:

1. Case (β_v). By definition of erasure, $\lceil (\lambda x.t)\ v \rceil$ is $(\lambda x.\lceil t \rceil)\ \lceil v \rceil$. Because the erasure of a value is a value, $\lceil v \rceil$ is a value; so, this term reduces in one step to $\lceil t \rceil [\lceil v \rceil / x]$. Because erasure commutes with substitution, this is $\lceil t[v/x] \rceil$. Thus, the first branch of the goal holds.
2. Case (*unfold/fold*). By definition of erasure $\lceil \mathsf{unfold}_n\, (\mathsf{fold}_n\, v) \rceil$ is $\lceil v \rceil$. Furthermore, $size(\mathsf{unfold}_n\, (\mathsf{fold}_n\, v))$ is $size(v) + 2$. Thus, the second branch of the goal holds.
3. Case (*context*). The hypothesis is $E[t] \longrightarrow E[t']$, where $t \longrightarrow t'$. Applying the induction hypothesis to $t \longrightarrow t'$ gives us:
   - Either $\lceil t \rceil \longrightarrow \lceil t' \rceil$. Then, by (repeated) evaluation under a context, we have $\lceil E \rceil [\lceil t \rceil] \longrightarrow \lceil E \rceil [\lceil t' \rceil]$, and because erasure commutes with context filling, this is $\lceil E[t] \rceil \longrightarrow \lceil E[t'] \rceil$. Thus, the first branch of the goal holds.
   - Or $\lceil t \rceil = \lceil t' \rceil$ and $size(t) > size(t')$. Then, by a similar argument, we have $\lceil E[t] \rceil = \lceil E[t'] \rceil$. Furthermore, we have $size(E[t]) = size(E) + size(t) > size(E) + size(t') = size(E[t'])$. Thus, the second branch of the goal holds.

The Forward Simulation lemma allows obtaining the two desired results:

- If $t \longrightarrow^\star v$ then we have $\lceil t \rceil \longrightarrow^\star \lceil v \rceil$. Because $\lceil v \rceil$ is a value, this is the first desired result.
- If out of the term t there is an infinite sequence S of reduction steps, then out of the term $\lceil t \rceil$ there is an infinite sequence S' of reduction steps and size-decreasing stuttering steps. Because every size-decreasing sequence must be finite, out of the sequence S' one can extract an infinite sequence of reduction steps. Therefore, the term $\lceil t \rceil$ diverges.

This proof is fundamentally the same as the proof of type erasure that was studied in the course concerning the type abstractions and type applications of System F.

Question 12

The sequence $1, 2$ is a sequence of two elements. The binary representation of "two" is 01, so we expect this sequence to have the shape `ZERO (ONE (_, NIL))`. By populating this shape with the integers 1 and 2, we find that the desired sequence is `ZERO (ONE ((1, 2), NIL))`.

It is important to note that the type `'a seq` is an unusual kind of algebraic data type: for example, if `ZERO ps` is a sequence of integers then `ps` must be a sequence of *pairs* of integers. This is known as a nested algebraic data type. The functions that manipulate such a data type usually exploit polymorphic recursion, so, in OCaml, they must carry a polymorphic type annotation.

The sequence $1, 2, 3$ is a sequence of three elements. The binary representation of "three" is 11, so we expect this sequence to have the shape `ONE (_, ONE (_, NIL))`. By populating this shape with the integers 1, 2, 3, we find that the desired sequence is `ONE (1, ONE ((2, 3), NIL))`.

The sequence $1, 2, 3, 4, 5, 6$ is a sequence of six elements. The binary expression of "six" is 011, so we expect this sequence to have the shape `ZERO (ONE (_, ONE (_, NIL)))`. By populating this shape with the integers 1, 2, 3, 4, 5, 6, we find that the desired sequence is `ZERO (ONE ((1, 2), ONE (((3, 4), (5, 6)), NIL)))`.

It is worth noting that, in each case, the correct answer is unique. This stems from the fact that `ZERO NIL` is forbidden. Thanks to this rule, there is a unique way of writing down a number in binary form, and once this shape has been determined, there is a unique way of populating it with the elements of the sequence. In other words, this representation of sequences is canonical. Two data structures of type `'a seq` are structurally equal if and only if they represent the same sequence of elements.

Question 13

Let us note that this problem and its solution are based on Chris Okasaki's book *Purely Functional Data Structures*, §10.1.2.

Because `cons` transforms a sequence of n elements into a sequence of $n+1$ elements, it is analogous to the incrementation of a number in binary notation. Adding 1 to the digit 0 results in the digit 1. Adding 1 to the digit 1 results in the digit 0 and requires a recursive call, which propagates a carry. In this case, two elements `x` and `y` are combined into a pair `(x, y)`, which at the next level of the data structure is considered as one element.

```
let rec cons : 'a . 'a -> 'a seq -> 'a seq = fun x xs ->
  match xs with
  | NIL ->
      ONE (x, NIL)
  | ZERO ps ->
      ONE (x, ps)
  | ONE (y, ps) ->
      ZERO (cons (x, y) ps)
```

In the last line, it is a mistake to write `ONE ((x, y), ps)` instead of `cons (x, y) ps`. Such a use of `ONE` is not well-typed: whereas `cons` expects an element and a sequence and produces a sequence, `ONE` expects an element and a sequence of pairs and produces a sequence of pairs. Furthermore, if such a use of `ONE` was possible, then `cons` would run in constant time, contradicting the intuition that a carry must be propagated.

To prove that this code never builds a sequence of the form `ZERO NIL`, it suffices to check, in the last branch, that the recursive call `cons (x, y) ps` cannot return `NIL`. This is obvious by visual inspection of the three branches. (One might also argue that `cons x xs` is a sequence of length $n+1$, therefore cannot be `NIL`. But this would require first proving that `cons` is correct, a more complex proof.)

Question 14

The code can be written as follows:

```
let rec uncons : 'a . 'a seq -> 'a * 'a seq = fun xs ->
  match xs with
  | NIL ->
      assert false
  | ONE (x, NIL) ->
      x, NIL
  | ONE (x, ps) ->
      x, ZERO ps
  | ZERO ps ->
      let (x, y), ps = uncons ps in
      x, ONE (y, ps)
```

The first branch cannot be taken because the sequence `xs` must be nonempty. (The recursive call `uncons ps` respects this requirement: there, `ps` must be nonempty because `xs` cannot be `ZERO NIL`.)

The other branches are obtained (roughly) by exchanging left-hand sides and right-hand sides in the definition of `cons`. For example, `cons x` transforms `ZERO ps` into `ONE (x, ps)`: here, conversely, `uncons` transforms `ONE (x, ps)` into a pair of `x` and `ZERO ps`.

To prove that this code never builds a sequence of the form `ZERO NIL`, it suffices to check, in the third branch, that `ps` cannot be `NIL`. This must be the case because the case where `ps` is `NIL` has been handled by the second branch in the definition of `uncons`.

Question 15

The cost of `cons` and `uncons` is the same as the cost of incrementing or decrementing a natural number in binary notation. Because of carry propagation, this cost is proportional in the worst case to the number of digits, that is, $O(\log n)$, where n is the length of the sequence, that is, the number of elements in the sequence.

Question 16

The code can be written as follows:

```
let rec get : 'a . int -> 'a seq -> 'a = fun i xs ->
  match xs with
  | NIL ->
      assert false
  | ZERO ps ->
      let x, y = get (i / 2) ps in
      if i mod 2 = 0 then x else y
  | ONE (x, ps) ->
      if i = 0 then x else get (i-1) (ZERO ps)
```

The first branch cannot be taken because the fact that the index `i` is valid implies that the sequence is nonempty.

In the second branch, the sequence `xs` has even length, say $2n$, and the subsequence `ps` is a sequence of n pairs. A recursive call extracts the pair `(x, y)` at index `i/2`. There remains to check whether `i` is even or odd so as to return `x` or `y`.

In the last branch, we first handle the case where `i` is zero. Then, we are back to the problem of looking up index `i-1` in a sequence of even length, as in the previous case. We temporarily build the sequence `ZERO ps`, where `ps` might be `NIL`. If this is considered inelegant then the last branch can be written under the equivalent form:

```
if i = 0 then x else
let i = i-1 in
let x, y = get (i / 2) ps in
if i mod 2 = 0 then x else y
```

This form is more verbose but offers the added benefit of saving one memory allocation, since the sequence `ZERO ps` is not constructed. Besides, the code duplication can be avoided by defining an auxiliary function `get_ZERO`, which is mutually recursive with `get`. This is left as an exercise to the reader.

Question 17

The code can be written as follows:

```
let rec set : 'a . int -> 'a -> 'a seq -> 'a seq = fun i e xs ->
  match xs with
  | NIL ->
      assert false
  | ZERO ps ->
      let x, y = get (i / 2) ps in
      let p = if i mod 2 = 0 then (e, y) else (x, e) in
      ZERO (set (i / 2) p ps)
  | ONE (x, ps) ->
      if i = 0 then ONE (e, ps)
      else cons x (set (i-1) e (ZERO ps))
```

In the second branch, we first read a pair `(x, y)` at index `i/2` in the subsequence `ps`; then, using a recursive call to `set`, we replace this pair with either `(e, y)` or `(x, e)`, depending on the parity of the index `i`.

In the last branch, on the last line, instead of `cons x (...)`, one may be tempted to write `ONE (x, set (i-1) e (ZERO ps))`. However, this does not make sense; it is not well-typed. In the code above, the recursive call `set (i-1) e (ZERO ps)` must return a sequence of the form `ZERO ps'`. What we want, then, is to construct the sequence `ONE (x, ps')`. The function `cons` is used for this purpose. Another solution, which does not use `cons`, is to write an auxiliary function `set_ZERO`, as follows:

```
let rec set : 'a . int -> 'a -> 'a seq -> 'a seq = fun i e xs ->
  match xs with
  | NIL ->
      assert false
  | ZERO ps ->
      ZERO (set_ZERO i e ps)
  | ONE (x, ps) ->
      if i = 0 then ONE (e, ps)
      else ONE (x, set_ZERO (i-1) e ps)

and set_ZERO : 'a. int -> 'a -> ('a * 'a) seq -> ('a * 'a) seq = fun i e ps ->
  let x, y = get (i / 2) ps in
  let p = if i mod 2 = 0 then (e, y) else (x, e) in
  set (i / 2) p ps
```

There remains to justify why this code cannot build a sequence of the form `ZERO NIL`. In the second version of the code, the constructor `ZERO` is used in one place, in the second branch of `set`. Thus, we must argue that `set_ZERO i e ps` cannot be `NIL`. This is the case because it is clear, by inspection of the code, that a sequence returned by `set` or `set_ZERO` must begin with `ZERO` or `ONE`.

One could also argue that the sequence `set i e xs` has exactly the same skeleton of constructors as the sequence `xs`. Therefore, if `xs` does not contain `ZERO NIL`, then `set i e xs` does not contain it either.

Question 18

In the worst case, `get` examines all of the digits in the sequence `xs`. At each digit, a constant amount of work is performed. Therefore the worst-case time complexity of `get` is $O(\log n)$, where n is the length of the sequence.

In the worst case, `set` examines all of the digits in the sequence `xs`. At each digit, a call to `get` is made. Therefore the worst-case time complexity of `get` is $O(\log^2 n)$. This is not satisfactory. A more efficient approach is to define `set` in terms of `transform`, which comes up next.

Question 19

The answer is simple: `let set i x xs = transform i (fun _ -> x) xs`.

Question 20

There are three λ-abstractions in the code that can flow into the formal parameter `f` of `transform`. One is the function `fun _ -> x` in the definition of `set` in terms of `transform`. The other two are the anonymous functions `fun (x, y) -> (f x, y)` and `fun (x, y) -> (x, f y)` inside `transform` itself. Let us use the names `K`, `L`, `R` to designate these three anonymous functions. This leads us to the following GADT definition:

```
type 'a transf =
  | K : 'a -> 'a transf
  | L : 'a transf -> ('a * 'a) transf
  | R : 'a transf -> ('a * 'a) transf
```

Because `f` is a function of type `'a -> 'a`, where the domain and codomain are the same, the type `'a transf` needs just one type parameter, in contrast with the type `('a, 'b) arrow` that was studied in the course.

One can then define `apply`:

```
let rec apply : type a . a transf -> a -> a = fun this that ->
  match this with
  | K x ->
      x
  | L f ->
      let (x, y) = that in
      (apply f x, y)
  | R f ->
      let (x, y) = that in
      (x, apply f y)
```

Finally, one defines `transform` and `set`:

```
let rec transform : 'a . int -> 'a transf -> 'a seq -> 'a seq =
fun i f xs ->
  match xs with
  | NIL ->
      assert false
  | ZERO ps ->
      ZERO (transform_ZERO i f ps)
  | ONE (x, ps) ->
      if i = 0 then ONE (apply f x, ps)
      else ONE (x, transform_ZERO (i-1) f ps)

and transform_ZERO : 'a. int -> 'a transf -> ('a * 'a) seq -> ('a * 'a) seq =
fun i f ps ->
  let f = if i mod 2 = 0 then L f else R f in
  transform (i / 2) f ps

let set i x xs =
  transform i (K x) xs
```

This is a special case where `apply` and `transform` are not mutually recursive.

Question 21

We analyze the defunctionalized variant of `transform` because it is clearer. Under the assumption that the OCaml compiler uses closure conversion or defunctionalization, the two variants have the same cost. (However, one should note that the non-defunctionalized variant is more general, as it can be applied to an arbitrary function `f`.)

A value `f` of type `'a transf` is just a list of letters `L` and `R`, terminated with a letter `K`. It can be understood as a path into a tree of pairs.

The time complexity of `apply f x` is clearly linear in the length of the path `f`.

The function `transform` traverses the digits in the sequence `xs`. The number of digits is $O(\log n)$, where n is the length of the sequence. On the way down, `transforms` builds a path `f` whose length is $O(\log n + f)$, where by f we mean the length of the path that is initially passed to `transform`. (The path is extended inside `transform_ZERO`, where `f` is replaced with `L f` or `R f`.) Once `transform` reaches the base case where `i` is zero, it calls `apply f x`. Because the length of the path `f` is $O(\log n + f)$, the cost of this call is $O(\log n + f)$. Therefore the overall cost of `transform` is $O(\log n) + O(\log n + f) = O(\log n + f)$.

In the definition of `set` in terms of `transform`, the path that is initially passed to `transform` is of the form `K x`; its length is 1. Therefore the cost of `set` is $O(\log n)$.